

Database Documentation

The Entity Relationship Diagram (ERD) for the Mobile Money sms processing system was designed to translate raw xml data into a scalable, normalized relational schema that guarantees data integrity and optimal query performance. The schema is organized around four core entities and a junction entity to handle complex transaction patterns.

Core Entities and cardinality

Users (Sender and receiver): stores unique customer profile information(Phone number, name account status). Because a user can initialise or receive multiple transactions over time, a 1:M(One to Many) relationship exists between Users and Transactions.

Transaction_categories: Classifies financial movements into distinct operational types(e.g Bank Transfer, cash Out, Airtime Top up, Utility payment). Each category can encompass many transactions(1:M), allowing easy aggregation for analytics.

System logs: Tracks data extraction, xml parsing status, dead letter snippet routing. And processing with Transaction to provide full auditability for every processed record.

Resolving Many to Many Relationships

A user can hold multiple roles across different transactions that can involve multiple parties (sender and receiver). To resolve potential M:N ambiguity, a junction/associative table named Transaction_Parties was introduced. This table links Users to Transactions via composite Foreign keys while specifying the exact participant role(Sender vs receiver) for each record.

Data integrity and constraints

Primary keys(e.g: transaction_id, user_id) utilize auto incrementing integers or UUID'S, while Foreign Key constraints enforce referential integrity across related tables. Check constraints prevent negative monetary amounts(amount >= 0), and standard Datetime fields ensure ISO-8601 compliance across all logged system events.

1. Data Dictionary

USERS

Column	Data Type	Description
user_id	INT	Unique ID of the user

Column	Data Type	Description
phone_number	VARCHAR(18)	Phone number of the user
full_name	VARCHAR(30)	Full name of the user
user_type	VARCHAR(12)	Shows whether the user is a customer or merchant

TRANSACTIONS

Column	Data Type	Description
transaction_id	INT	Unique ID of the transaction
momo_tx_id	VARCHAR(50)	Mobile Money transaction ID
sender_id	INT	ID of the user sending the money
receiver_id	INT	ID of the user receiving the money
amount	DECIMAL(10,2)	Amount of money involved in the transaction
fee	DECIMAL(10,2)	Fee charged for the transaction
time_stamp	DATETIME	Date and time when the transaction happened

SYSTEM_LOGS

Column	Data Type	Description
log_id	INT	Unique ID for the log
transaction_id	INT	ID of the transaction related to the log
log_level	INT	Shows the level of the system log
message	TEXT	Message describing what happened

TRANSACTION_CATEGORIES

Column	Data Type	Description
category_id	INT	Unique ID of the category
category_name	VARCHAR(20)	Name of the transaction category

TRANSACTION_CATEGORY_MAPPINGS

Column	Data Type	Description
transaction_id	INT	ID of the transaction
category_id	INT	ID of the category

This table connects transactions with their categories.

2. Sample Queries

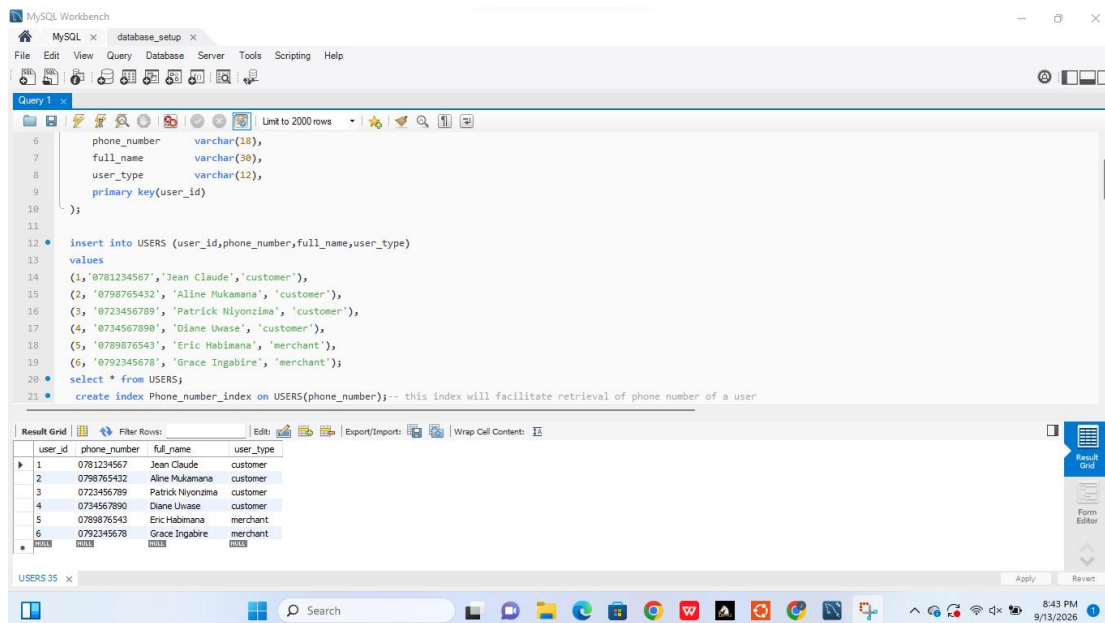
The following queries demonstrate some of the functionality of the database.

Display all users

```
SELECT * FROM USERS;
```

This displays all users stored in the USERS table.

Screenshot:



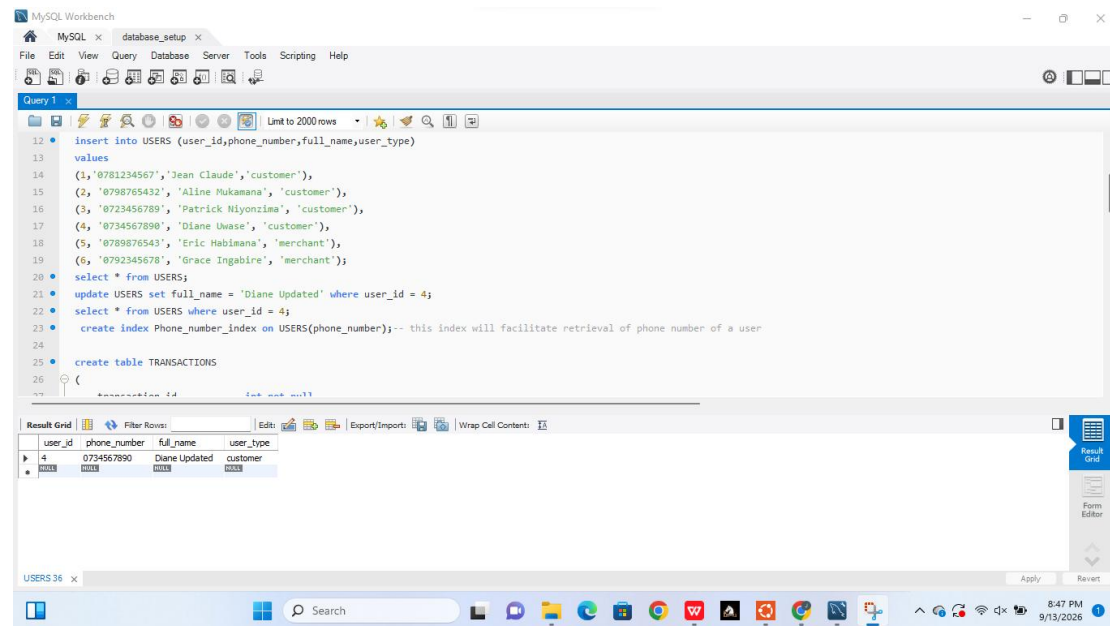
Find one user

```
SELECT * FROM USERS
```

```
WHERE user_id = 4;
```

This displays the user whose ID is 4.

Screenshot:

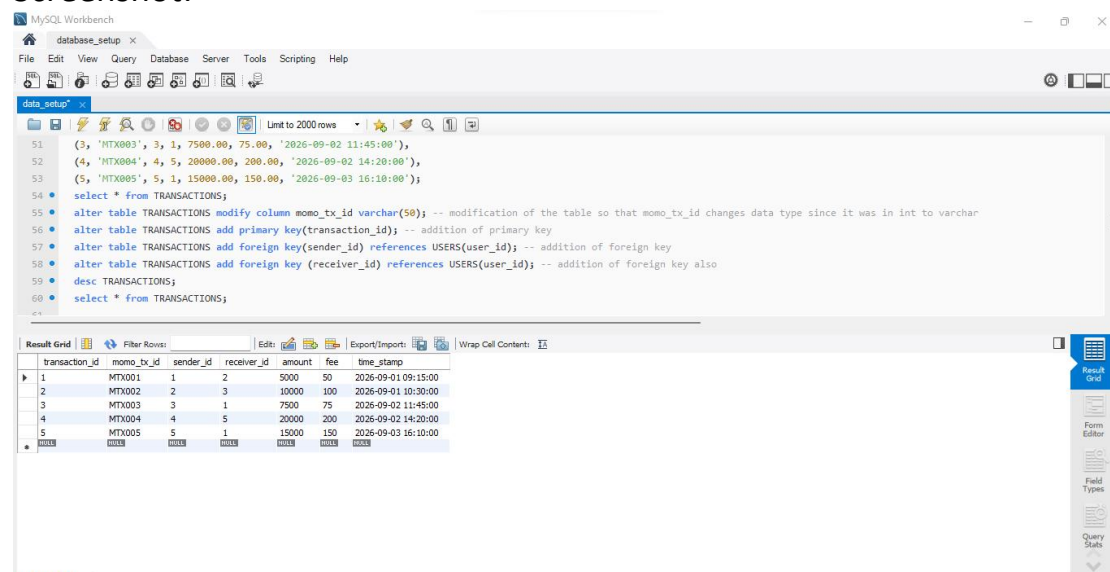


Display transactions

SELECT * FROM TRANSACTIONS;

This displays all the transactions stored in the database.

Screenshot:



Find transactions made by a specific sender

SELECT * FROM TRANSACTIONS

WHERE sender_id = 1;

This displays transactions where user 1 was the sender.

Update a user

UPDATE USERS

SET full_name = 'Diane Uwase'

WHERE user_id = 4;

This changes the name of the user with ID 4.

Delete a user

DELETE FROM USERS

WHERE user_id = 7;

This removes the user with ID 7.

View transaction categories

SELECT * FROM TRANSACTION_CATEGORIES;

This displays all the available transaction categories.

3. Rules Added for Security and Accuracy

The database includes several rules to help keep the data accurate and maintain relationships between tables.

Primary keys

Primary keys make sure that each record can be uniquely identified.

The following columns are primary keys:

USERS: user_id

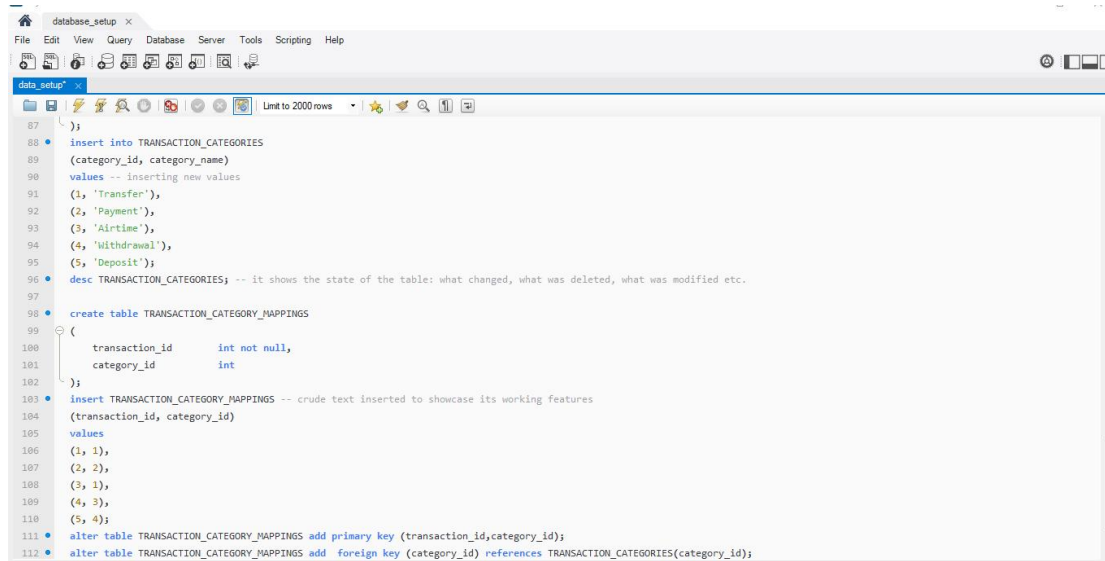
TRANSACTIONS: transaction_id

SYSTEM_LOGS: log_id

TRANSACTION_CATEGORIES: category_id

The TRANSACTION_CATEGORY_MAPPINGS table uses transaction_id and category_id together as a primary key.

Screenshot:



DESC USERS;

You can also use:

DESC TRANSACTIONS;

Foreign keys

Foreign keys make sure that related records exist in the database.

The TRANSACTIONS table has:

FOREIGN KEY(sender_id) REFERENCES USERS(user_id);

FOREIGN KEY(receiver_id) REFERENCES USERS(user_id);

The SYSTEM_LOGS table has:

FOREIGN KEY(transaction_id) REFERENCES

TRANSACTIONS(transaction_id);

The TRANSACTION_CATEGORY_MAPPINGS table has:

FOREIGN KEY(category_id)

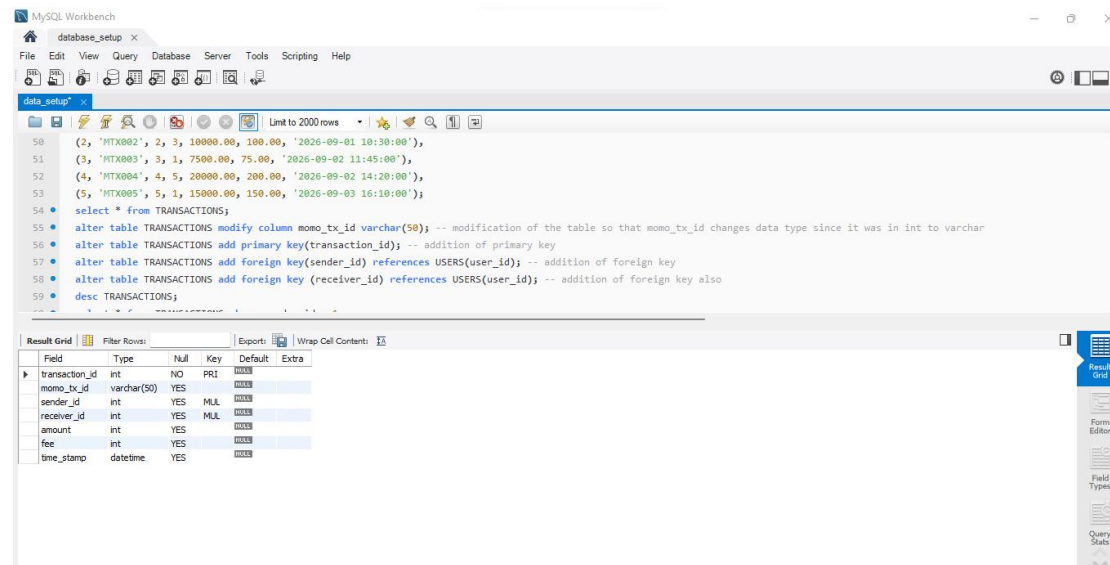
REFERENCES TRANSACTION_CATEGORIES(category_id);

FOREIGN KEY(transaction_id)

REFERENCES TRANSACTIONS(transaction_id);

These rules help prevent transactions or logs from referring to users or transactions that do not exist.

Screenshot:



Amount and fee checks

The database prevents invalid transaction amounts.

The amount must be greater than zero:

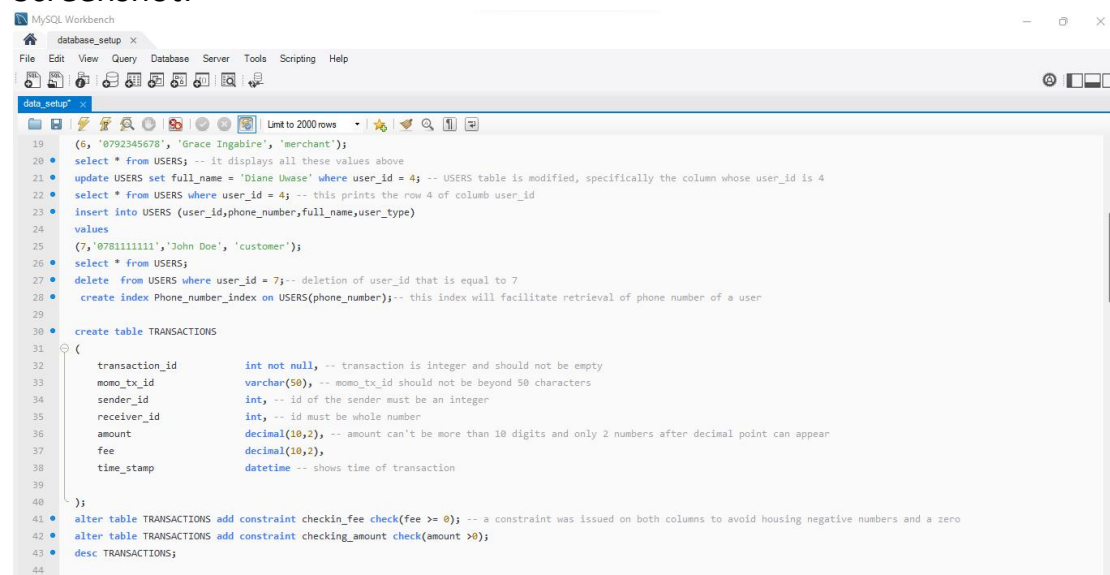
CHECK(amount > 0);

The fee cannot be negative:

CHECK(fee >= 0);

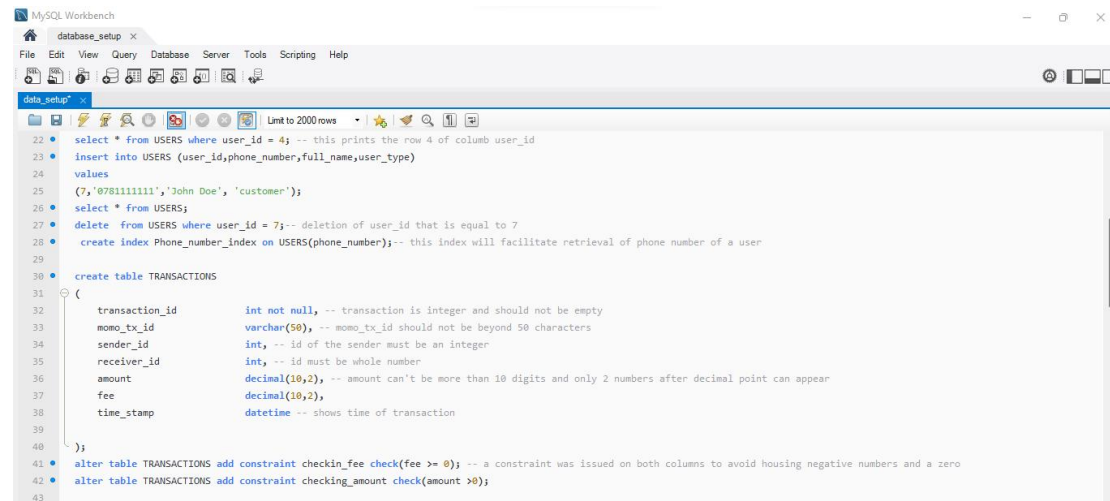
These rules help prevent incorrect financial information from being stored.

Screenshot:



SHOW CREATE TABLE TRANSACTIONS;

Screenshot:

A screenshot of the MySQL Workbench interface. The 'Query' tab is active, showing a SQL script. The script includes a select statement for user_id 4, an insert into the USERS table, a delete from the USERS table, a create index statement for Phone_number_index, and a create table statement for TRANSACTIONS. The TRANSACTIONS table has columns: transaction_id (int not null), momo_tx_id (varchar(50)), sender_id (int), receiver_id (int), amount (decimal(10,2)), fee (decimal(10,2)), and time_stamp (datetime). It also includes two alter table statements to add check constraints on the fee and amount columns.

```
22 • select * from USERS where user_id = 4; -- this prints the row 4 of columb user_id
23 • insert into USERS (user_id,phone_number,full_name,user_type)
24 values
25 (7,'0781111111','John Doe', 'customer');
26 • select * from USERS;
27 • delete from USERS where user_id = 7;-- deletion of user_id that is equal to 7
28 • create index Phone_number_index on USERS(phone_number);-- this index will facilitate retrieval of phone number of a user
29
30 • create table TRANSACTIONS
31 (
32     transaction_id          int not null, -- transaction is integer and should not be empty
33     momo_tx_id              varchar(50), -- momo_tx_id should not be beyond 50 characters
34     sender_id               int, -- id of the sender must be an integer
35     receiver_id             int, -- id must be whole number
36     amount                  decimal(10,2), -- amount can't be more than 10 digits and only 2 numbers after decimal point can appear
37     fee                     decimal(10,2),
38     time_stamp              datetime -- shows time of transaction
39 )
40 );
41 • alter table TRANSACTIONS add constraint checkin_fee check(fee >= 0); -- a constraint was issued on both columns to avoid housing negative numbers and a zero
42 • alter table TRANSACTIONS add constraint checking_amount check(amount >0);
43
```

NOT NULL rules

Some important ID columns were created with NOT NULL.

For example:

user_id INT NOT NULL

And transaction_id INT NOT NULL

This prevents these important IDs from being empty.

Phone number index

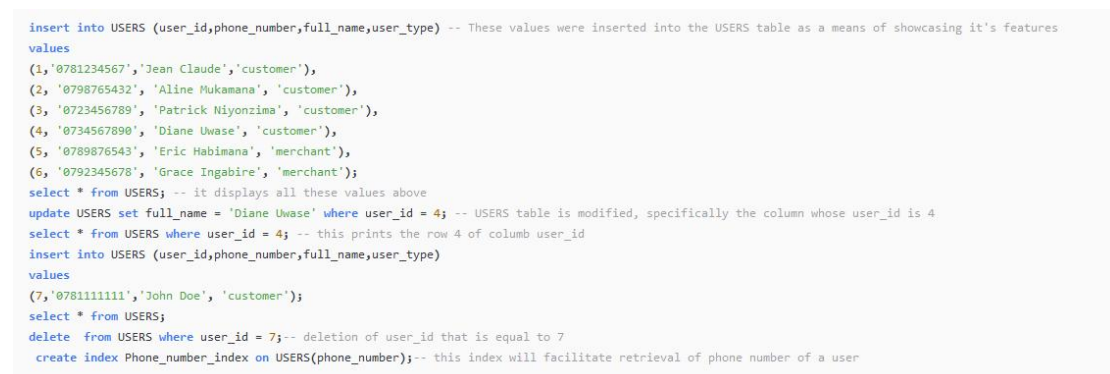
An index was created on the phone number column:

CREATE INDEX Phone_number_index

ON USERS(phone_number);

This helps improve the speed of searches using phone numbers.

Screenshot:

A screenshot of the MySQL Workbench interface showing a SQL script. The script includes an insert into the USERS table with six rows of data, a select statement to display all values, an update statement to modify the full_name of user_id 4, another select statement to print the row 4, another insert into the USERS table, and a create index statement for Phone_number_index.

```
insert into USERS (user_id,phone_number,full_name,user_type) -- These values were inserted into the USERS table as a means of showcasing it's features
values
(1,'0781234567','Jean Claude','customer'),
(2,'0798765432','Aline Mukamana','customer'),
(3,'0723456789','Patrick Niyonzima','customer'),
(4,'0734567890','Diane Uwase','customer'),
(5,'0789876543','Eric Habimana','merchant'),
(6,'0792345678','Grace Ingabire','merchant');
select * from USERS; -- it displays all these values above
update USERS set full_name = 'Diane Uwase' where user_id = 4; -- USERS table is modified, specifically the column whose user_id is 4
select * from USERS where user_id = 4; -- this prints the row 4 of columb user_id
insert into USERS (user_id,phone_number,full_name,user_type)
values
(7,'0781111111','John Doe', 'customer');
select * from USERS;
delete from USERS where user_id = 7;-- deletion of user_id that is equal to 7
create index Phone_number_index on USERS(phone_number);-- this index will facilitate retrieval of phone number of a user
```

Conclusion

The database uses primary keys, foreign keys, NOT NULL rules, CHECK constraints, and an index to improve data accuracy and make the

database easier to use. The sample queries also demonstrate how users and transactions can be added, viewed, updated, and deleted.